CIT 4409: NETWORK APPLICATION PROGRAMMING

MODULE TWO

Week Two: TCP/IP for Programmers

Topic: IP, Ports, Sockets, Process Communication, Python Socket Basics

Delivery Mode: Lecture & Lab

Practical Project / Lab Work: Python socket basics

1.0 Introduction

The TCP/IP protocol suite is the foundation of all network communications. Understanding TCP/IP is critical for network programmers because it allows applications to send and receive data over networks reliably.

This lecture covers:

1. Internet Protocol (IP) addressing
2. Ports and their significance
3. Socket programming concepts
4. Inter-process communication (IPC) over networks
5. Python socket basics for practical implementations

By the end of this lecture, students should be able to:

- Understand IP addressing and TCP/UDP distinctions
- Identify port numbers and their roles in networking
- Use sockets to create client-server programs in Python
- Implement basic process communication over TCP/IP

2.0 TCP/IP Protocol Suite Overview

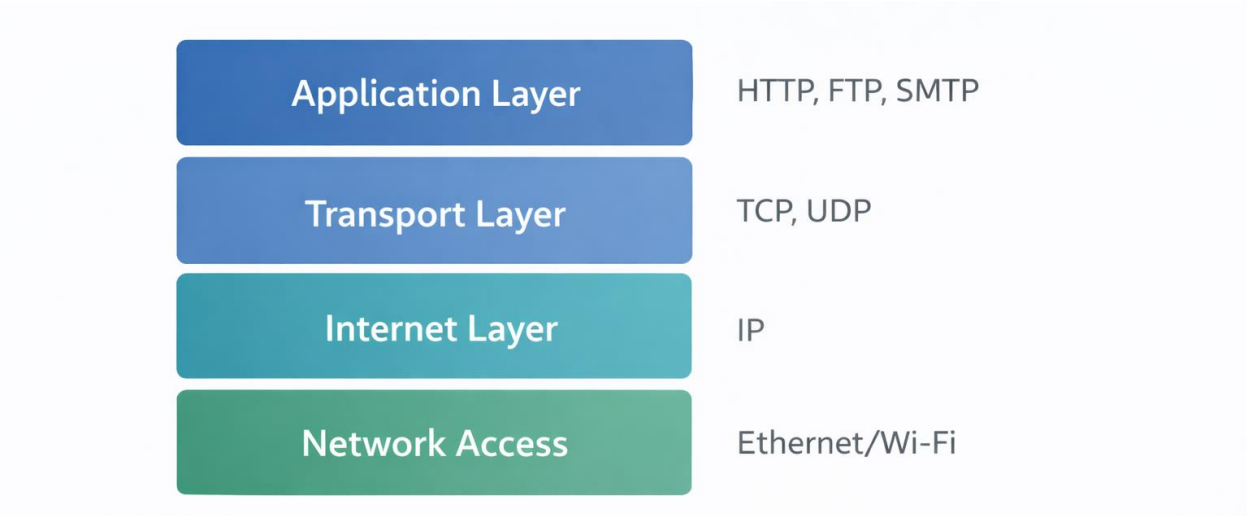
2.1 What is TCP/IP?

- **TCP/IP** stands for Transmission Control Protocol/Internet Protocol.
- It is a layered suite of protocols for communication across networks.

Layered Model:

Layer	Protocol	Function
Application	HTTP, FTP, SMTP	Network services for applications
Transport	TCP, UDP	Process-to-process communication
Internet	IP	Routing packets between hosts
Network Access	Ethernet, Wi-Fi	Physical transmission of data

Diagram 1 – TCP/IP Stack:



2.2 TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable, guarantees delivery	Unreliable, no guarantee
Ordering	Guarantees order	No ordering
Speed	Slower	Faster
Use Cases	Web, Email, File Transfer	Video Streaming, VoIP, Games

Example:

- Web browsing uses TCP (HTTP/HTTPS)
- Online gaming often uses UDP

3.0 IP Addresses

3.1 What is an IP Address?

An IP address uniquely identifies a device on a network.

- IPv4: 32-bit address, e.g., 192.168.1.10
- IPv6: 128-bit address, e.g., 2001:0db8:85a3::8a2e:0370:7334

Structure of IPv4:

- **Network ID:** Identifies the network
- **Host ID:** Identifies the device on the network

Diagram 2 – IPv4 Address Layout:

3.2 Public vs Private IP Addresses

Type	Range (IPv4)	Use
Private	10.0.0.0 – 10.255.255.255	Internal LAN
Private	172.16.0.0 – 172.31.255.255	Internal LAN
Private	192.168.0.0 – 192.168.255.255	Home networks
Public	Any outside private ranges	Internet

Key Concept: NAT (Network Address Translation) allows private IPs to access the Internet.

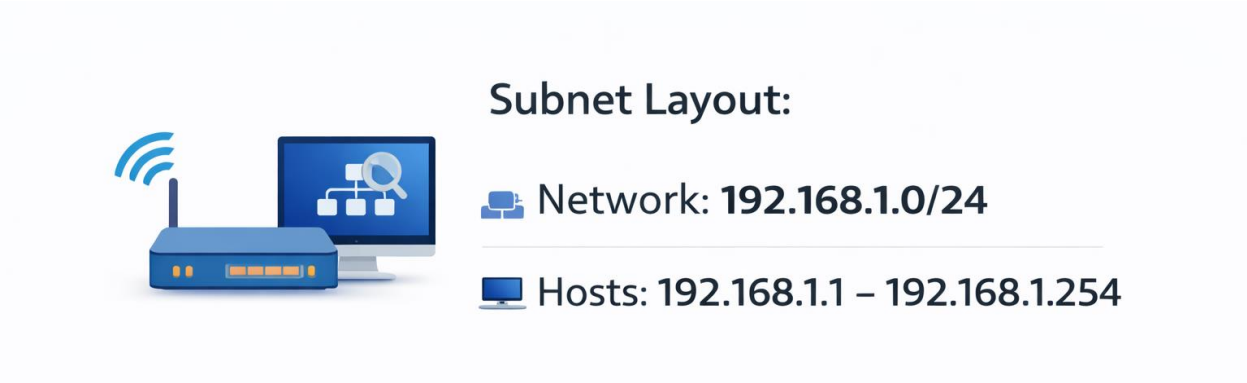
3.3 Subnetting (Basic)

Subnetting divides a network into smaller sub-networks for efficient routing.

Example:

- IP: 192.168.1.0
- Subnet Mask: 255.255.255.0
- Hosts: 254

Diagram 3 – Subnet Layout:



4.0 Ports and Process Communication

4.1 What is a Port?

A **port** identifies a process on a device.

- TCP/IP allows multiple processes to communicate simultaneously on a single host.
- **Port numbers range:** 0 – 65535
    - Well-known ports: 0 – 1023
    - Registered ports: 1024 – 49151
    - Dynamic/private: 49152 – 65535

Examples:

- HTTP: Port 80
- HTTPS: Port 443
- SMTP: Port 25
- FTP: Port 21

4.2 Socket Concept

A **socket** is the combination of an IP address and a port number.

- TCP/IP Socket = IP + Port + Protocol

Example:

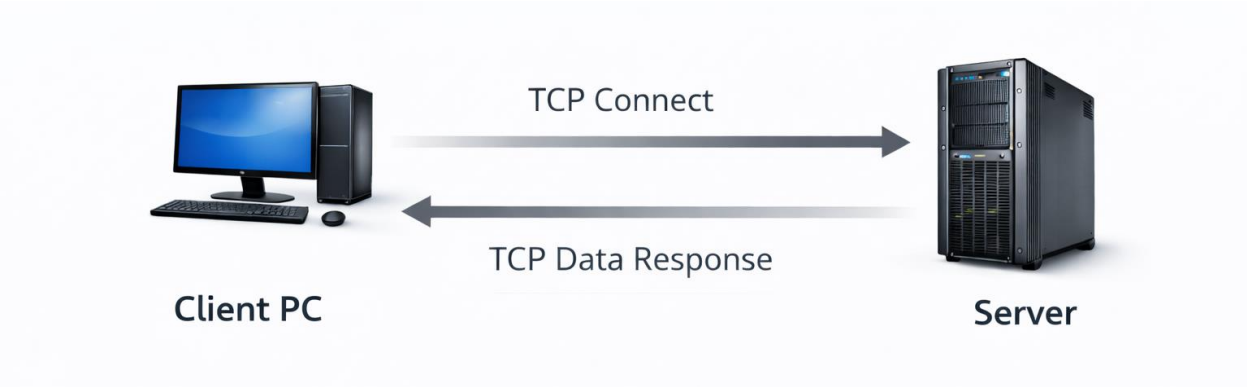
- 192.168.1.10:80 (Web Server)

Diagram 4 – Sockets:

4.3 Client/Server Communication

1. **Server** binds to an IP and port and listens for connections.
2. **Client** connects to server IP and port.
3. **Data exchange** occurs via TCP/UDP sockets.

Diagram 5 – Client/Server Communication:



5.0 Python Socket Basics

Python provides a **socket module** for network programming.

```python
import socket

# Create a socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # TCP

# Connect to a server
s.connect(('example.com', 80))

# Send data
s.sendall(b'GET / HTTP/1.1\r\nHost: example.com\r\n\r\n')

# Receive response
response = s.recv(4096)
print(response.decode())

# Close connection
s.close()
```

5.1 Creating a TCP Server in Python

```python
import socket

# Server setup
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('0.0.0.0', 12345))
server_socket.listen(5)
print("Server listening on port 12345")

while True:
    client_socket, addr = server_socket.accept()
    print(f"Connection from {addr}")
    client_socket.send(b"Hello from server!")
    client_socket.close()
```

Diagram 6 – Python TCP Server Flow:

[Server Socket] -> Listen -> Accept -> Send/Receive -> Close

5.2 Python TCP Client

```python
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 12345))
message = client_socket.recv(1024)
print(f"Received: {message.decode()}")
client_socket.close()
```

5.3 UDP Example

UDP Server:

```
import socket

udp_server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
udp_server.bind(('0.0.0.0', 54321))

while True:
    data, addr = udp_server.recvfrom(1024)
    print(f"Received from {addr}: {data.decode()}")
    udp_server.sendto(b'ACK', addr)
```

UDP Client:

```
import socket

udp_client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
udp_client.sendto(b'Hello Server', ('localhost', 54321))
response, addr = udp_client.recvfrom(1024)
print(f"Received: {response.decode()}")
```

6.0 Practical Exercises

1. Write a Python TCP client-server program that exchanges text messages.
2. Modify the server to handle **multiple clients using threading**.
3. Implement a UDP messaging app and observe differences in reliability.
4. Use socket.gethostbyname('example.com') to resolve hostnames to IPs.

Discussion Questions:

- Why is TCP reliable but slower than UDP?
- How do sockets allow multiple applications to communicate simultaneously?
- How does a server differentiate between clients?

7.0 Advanced Concepts (Optional Discussion)

- **Non-blocking sockets** in Python (socket.setblocking(False))
- **Select/poll** for monitoring multiple sockets
- **Socket options:** SO_REUSEADDR, SO_KEEPALIVE
- **Socket programming in real-world applications:** Web servers, chat apps, IoT communication

8.0 References

- Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7th ed.). Pearson.
- Rhodes, B., & Goerzen, J. (2014). *Foundations of Python Network Programming*.
- Ciubotaru, B., & Muntean, G.-M. (2013). *Advanced Network Programming*. Springer.
- Python Official Docs – https://docs.python.org/3/library/socket.html